

온라인 쇼핑몰 API 명세서 v1

0. 문서 목적

이 문서는 Spring Boot 기반 온라인 쇼핑몰의 REST API 명세를 정의한다.

API 설계의 핵심 목표는 다음과 같다.

1. 사용자 API와 관리자 API를 분리한다.
2. 주문, 결제, 재고의 상태 변경 API는 멍등성과 동시성을 고려한다.
3. 목록 조회 API는 cursor 기반 페이징을 기본으로 한다.
4. 모든 응답은 일관된 포맷을 사용한다.
5. 오류 응답은 운영자가 추적할 수 있도록 errorCode를 포함한다.

1. 공통 규칙

1-1. Base URL

`/api`

관리자 API:

`/api/admin`

1-2. 인증 방식

사용자 인증이 필요한 API는 다음 헤더를 사용한다.

`Authorization: Bearer {accessToken}`

관리자 API는 ADMIN 권한이 필요하다.

1-3. 공통 응답 포맷

성공 응답:

```
{
  "success": true,
  "data": {},
  "error": null
}
```

실패 응답:

```
{
  "success": false,
  "data": null,
  "error": {
    "code": "OUT_OF_STOCK",
    "message": "재고가 부족합니다.",
    "traceId": "a1b2c3d4"
  }
}
```

1-4. 공통 에러 코드

코드	설명
INVALID_REQUEST	요청 값이 올바르지 않음
UNAUTHORIZED	인증 필요
FORBIDDEN	권한 없음
NOT_FOUND	리소스 없음
OUT_OF_STOCK	재고 부족
DUPLICATE_REVIEW	이미 리뷰 작성됨
INVALID_ORDER_STATUS	주문 상태가 올바르지 않음
PAYMENT_FAILED	결제 실패
DUPLICATE_PAYMENT	중복 결제 요청
EXTERNAL_API_ERROR	외부 API 오류
INTERNAL_SERVER_ERROR	서버 내부 오류

1-5. Cursor Pagination 응답

목록 조회는 기본적으로 cursor 기반 페이지ング을 사용한다.

```
{
  "items": [],
  "nextCursor": "2026-06-28T12:00:00_100",
  "hasNext": true
}
```

2. 회원 API

2-1. 회원가입

`POST /api/members`

Request

```
{
  "email": "user@example.com",
  "password": "password1234!",
  "name": "홍길동",
  "phone": "010-1234-5678"
}
```

Response

```
{
  "success": true,
  "data": {
    "memberId": 1,
    "email": "user@example.com",
    "name": "홍길동"
  },
  "error": null
}
```

검증 규칙

- email은 UNIQUE
- password는 BCrypt로 암호화 저장
- 동일 이메일 가입 불가

2-2. 로그인

POST /api/login

Request

```
{
  "email": "user@example.com",
  "password": "password1234!"
}
```

Response

```
{
  "success": true,
  "data": {
    "accessToken": "access-token",
    "refreshToken": "refresh-token"
  },
  "error": null
}
```

2-3. 내 정보 조회

GET /api/members/me

Response

```
{
  "success": true,
  "data": {
    "memberId": 1,
    "email": "user@example.com",
    "name": "홍길동",
    "phone": "010-****-5678"
  },
  "error": null
}
```

3. 상품 API

3-1. 상품 목록 조회

```
GET /api/products?categoryId=1&sort=latest&cursor=2026-06-28T12:00:00_100&size=20
```

Query Parameters

이름	필수	설명
categoryId	N	카테고리 ID
sort	N	latest, priceAsc, priceDesc
cursor	N	다음 페이지 커서
size	N	조회 개수, 기본 20

Response

```
{
  "success": true,
  "data": {
    "items": [
      {
        "productId": 100,
        "name": "무선 마우스",
        "price": 29000,
        "thumbnailUrl": "https://image.example.com/mouse.png",
        "reviewCount": 120,
        "averageRating": 4.7,
        "status": "SELLING"
      }
    ],
    "nextCursor": "2026-06-28T11:50:00_99",
    "hasNext": true
  },
  "error": null
}
```

성능 기준

- Entity 직접 노출 금지
- 목록에 필요한 컬럼만 DTO로 조회
- category_id, status, created_at, product_id 복합 인덱스 사용

3-2. 상품 상세 조회

```
GET /api/products/{productId}
```

Response

```
{
  "success": true,
  "data": {
    "productId": 100,
    "name": "무선 마우스",
    "price": 29000,
    "description": "인체공학 무선 마우스",
    "thumbnailUrl": "https://image.example.com/mouse.png",
    "status": "SELLING",
    "availableQuantity": 30,
    "reviewCount": 120,
    "averageRating": 4.7
  },
  "error": null
}
```

캐시 정책

- 상품 상세는 캐시 가능
- 상품 수정 시 캐시 무효화
- 재고 수량은 캐시하지 않거나 짧은 TTL 적용

4. 장바구니 API

4-1. 장바구니 상품 추가

```
POST /api/cart/items
```

Request

```
{
  "productId": 100,
  "quantity": 2
}
```

Response

```
{
  "success": true,
  "data": {
    "cartItemId": 10,
    "productId": 100,
    "quantity": 2
  },
  "error": null
}
```

규칙

- 동일 상품이 이미 있으면 수량 증가
- 상품 상태가 SELLING이 아니면 추가 불가

4-2. 장바구니 조회

```
GET /api/cart
```

Response

```
{
  "success": true,
  "data": {
    "cartId": 1,
    "items": [
      {
        "cartItemId": 10,
        "productId": 100,
        "productName": "무선 마우스",
        "price": 29000,
        "quantity": 2,
        "availableQuantity": 30
      }
    ]
  },
  "error": null
}
```

5. 주문 API

5-1. 주문 생성

POST /api/orders

Request

```
{
  "items": [
    {
      "productId": 100,
      "quantity": 2
    },
    {
      "productId": 200,
      "quantity": 1
    }
  ],
  "deliveryAddress": {
    "receiverName": "홍길동",
    "receiverPhone": "010-1234-5678",
    "zipcode": "05000",
    "address1": "서울시 광진구 ...",
    "address2": "101동 1001호"
  },
  "couponIssued": null,
  "usePointAmount": 0
}
```

Response

```
{
  "success": true,
  "data": {
    "orderId": 500,
    "orderNo": "ORD-20260628-000001",
    "orderStatus": "PENDING_PAYMENT",
    "totalPrice": 87000,
    "discountAmount": 0,
    "finalPaymentAmount": 87000,
    "paymentReady": {
      "paymentKey": "pay_abc123",
      "idempotencyKey": "ORD-20260628-000001-1"
    }
  }
}
```



```
}  
},  
"error": null  
}
```

처리 규칙

1. 회원 존재 여부 확인
2. 상품 존재 여부 확인
3. 상품 판매 상태 확인
4. product_id 오름차순으로 재고 예약
5. 주문 상태 PENDING_PAYMENT 생성
6. 주문상품 스냅샷 저장
7. 배송 정보 저장
8. 결제 준비 정보 생성

트랜잭션 범위

DB 트랜잭션 안에서 처리:

- 재고 예약
- 주문 생성
- 주문상품 생성
- 배송정보 생성
- 결제 READY 생성

DB 트랜잭션 밖에서 처리:

- 외부 PG 결제창 호출
- 이메일 발송
- SMS 발송

5-2. 내 주문 목록 조회

```
GET /api/orders?cursor=2026-06-28T12:00:00_500&size=20
```

Response

```
{  
  "success": true,  
  "data": {  
    "items": [  

```

```
{
  "orderId": 500,
  "orderNo": "ORD-20260628-000001",
  "orderStatus": "PAID",
  "totalPrice": 87000,
  "orderedAt": "2026-06-28T12:00:00",
  "mainProductName": "무선 마우스",
  "itemCount": 2
},
{
  "nextCursor": "2026-06-27T11:00:00_499",
  "hasNext": true
},
{
  "error": null
}
```

성능 기준

- member_id, ordered_at, order_id 복합 인덱스 사용
- Offset Pagination 지양
- 목록에서는 상세 주문상품 전체를 즉시 로딩하지 않음

5-3. 주문 상세 조회

```
GET /api/orders/{orderId}
```

Response

```
{
  "success": true,
  "data": {
    "orderId": 500,
    "orderNo": "ORD-20260628-000001",
    "orderStatus": "PAID",
    "totalPrice": 87000,
    "discountAmount": 0,
    "finalPaymentAmount": 87000,
    "items": [
      {
        "orderItemId": 1,
        "productId": 100,
        "productName": "무선 마우스",
        "orderPrice": 29000,

```

```

    "quantity": 2
  },
  "payment": {
    "paymentStatus": "APPROVED",
    "approvedAmount": 87000,
    "approvedAt": "2026-06-28T12:01:00"
  },
  "delivery": {
    "deliveryStatus": "READY",
    "receiverName": "홍길동",
    "receiverPhone": "010-****-5678",
    "address": "서울시 광진구 ..."
  }
},
"error": null
}

```

보안 규칙

- 일반 사용자는 본인 주문만 조회 가능
- 관리자만 전체 주문 조회 가능

5-4. 주문 취소

`POST /api/orders/{orderId}/cancel`

Request

```

{
  "reason": "단순 변심"
}

```

Response

```

{
  "success": true,
  "data": {
    "orderId": 500,
    "orderStatus": "CANCELLED",
    "cancelledAt": "2026-06-28T13:00:00"
  },
}

```

```
"error": null
}
```

처리 규칙

1. 본인 주문인지 확인
2. 주문 상태 확인
3. 배송 상태 확인
4. 배송 시작 전이면 취소 가능
5. 결제 승인 상태이면 PG 결제 취소 요청
6. 결제 취소 성공 시 주문 CANCELLED 변경
7. 재고 복구
8. 주문 이벤트 로그 저장

취소 불가 조건

- 이미 배송 중
- 이미 배송 완료
- 이미 취소됨
- 환불 프로세스로 넘어간 주문

6. 결제 API

6-1. 결제 승인

POST /api/payments/approve

Headers

Idempotency-Key: ORD-20260628-000001-1

Request

```
{
  "orderId": 500,
  "paymentKey": "pay_abc123",
  "amount": 87000
}
```

Response

```
{
  "success": true,
  "data": {
    "paymentId": 900,
    "orderId": 500,
    "paymentStatus": "APPROVED",
    "orderStatus": "PAID",
    "approvedAmount": 87000,
    "approvedAt": "2026-06-28T12:01:00"
  },
  "error": null
}
```

처리 규칙

1. Idempotency-Key 확인
2. 이미 승인된 결제면 기존 결과 반환
3. 주문 상태 PENDING_PAYMENT 확인
4. 결제 금액과 주문 금액 일치 확인
5. PG 승인 API 호출
6. payment APPROVED 저장
7. order PAID 변경
8. 예약 재고 확정
9. outbox_event 저장

6-2. PG 콜백 수신

```
POST /api/payments/callback
```

Request

```
{
  "paymentKey": "pay_abc123",
  "pgTransactionId": "pg_tx_123",
  "status": "APPROVED",
  "approvedAmount": 87000,
  "approvedAt": "2026-06-28T12:01:00"
}
```

Response

```
{
  "success": true,
  "data": {
    "received": true
  },
  "error": null
}
```

처리 규칙

- 콜백은 중복 수신될 수 있다.
- 이미 APPROVED 상태이면 무시하고 성공 응답한다.
- payment_key 기준으로 결제 정보를 찾는다.
- 주문 상태와 결제 상태가 불일치하면 보정한다.

6-3. 결제 취소

POST /api/payments/{paymentId}/cancel

Headers

Idempotency-Key: CANCEL-ORD-20260628-000001-1

Request

```
{
  "cancelAmount": 87000,
  "reason": "주문 취소"
}
```

Response

```
{
  "success": true,
  "data": {
    "paymentId": 900,
    "paymentStatus": "CANCELLED",
    "cancelledAmount": 87000,
    "cancelledAt": "2026-06-28T13:00:00"
  }
}
```

```
},  
"error": null  
}
```

7. 리뷰 API

7-1. 리뷰 작성

POST /api/products/{productId}/reviews

Request

```
{  
  "orderId": 1,  
  "rating": 5,  
  "content": "배송도 빠르고 제품도 좋아요."  
}
```

Response

```
{  
  "success": true,  
  "data": {  
    "reviewId": 300,  
    "productId": 100,  
    "rating": 5,  
    "content": "배송도 빠르고 제품도 좋아요.",  
    "createdAt": "2026-06-28T14:00:00"  
  },  
  "error": null  
}
```

처리 규칙

1. 로그인 회원 확인
2. order_item이 본인 주문인지 확인
3. 주문 상태가 DELIVERED 또는 CONFIRMED인지 확인
4. 해당 order_item에 리뷰가 이미 있는지 확인
5. review INSERT
6. product_stat 갱신 이벤트 발행

최종 방어선

```
review.order_item_id UNIQUE
```

7-2. 상품 리뷰 목록 조회

```
GET /api/products/{productId}/reviews?cursor=2026-06-28T12:00:00_300&size=20
```

Response

```
{
  "success": true,
  "data": {
    "items": [
      {
        "reviewId": 300,
        "memberName": "홍*동",
        "rating": 5,
        "content": "배송도 빠르고 제품도 좋아요.",
        "createdAt": "2026-06-28T14:00:00"
      }
    ],
    "nextCursor": "2026-06-27T10:00:00_299",
    "hasNext": true
  },
  "error": null
}
```

8. 관리자 API

8-1. 관리자 주문 검색

```
GET /api/admin/orders?status=PAID&from=2026-06-01&to=2026-06-28&cursor=500&size=50
```

Response

```
{
  "success": true,
  "data": {
    "items": [
```



```
{
  "orderId": 500,
  "orderNo": "ORD-20260628-000001",
  "memberEmail": "user@example.com",
  "orderStatus": "PAID",
  "totalPrice": 87000,
  "orderedAt": "2026-06-28T12:00:00"
},
{
  "nextCursor": "499",
  "hasNext": true
},
{
  "error": null
}
```

8-2. 상품 등록

POST /api/admin/products

Request

```
{
  "categoryId": 1,
  "name": "무선 마우스",
  "price": 29000,
  "description": "인체공학 무선 마우스",
  "thumbnailUrl": "https://image.example.com/mouse.png",
  "initialQuantity": 100
}
```

Response

```
{
  "success": true,
  "data": {
    "productId": 100,
    "inventory": {
      "totalQuantity": 100,
      "availableQuantity": 100,
      "reservedQuantity": 0
    }
  },
}
```

```
"error": null
}
```

8-3. 상품 대량 업로드

```
POST /api/admin/products/bulk-upload
```

Request

```
Content-Type: multipart/form-data
```

```
file=products.csv
```

Response

```
{
  "success": true,
  "data": {
    "uploadId": "upload_20260628_001",
    "status": "ACCEPTED"
  },
  "error": null
}
```

처리 흐름

1. CSV 업로드
2. staging_product 적재
3. 검증 배치 실행
4. 정상 데이터 product 반영
5. 실패 데이터 error 테이블 저장

9. 배치/운영 API

9-1. 배치 실행 이력 조회

```
GET /api/admin/batch-jobs?jobName=PaymentReconciliationJob&size=20
```

Response

```
{
  "success": true,
  "data": {
    "items": [
      {
        "executionId": 1000,
        "jobName": "PaymentReconciliationJob",
        "status": "COMPLETED",
        "processedCount": 100,
        "successCount": 100,
        "failCount": 0,
        "startedAt": "2026-06-28T03:00:00",
        "endedAt": "2026-06-28T03:01:00"
      }
    ]
  },
  "error": null
}
```

9-2. 배치 재실행

`POST /api/admin/batch-jobs/{executionId}/retry`

Response

```
{
  "success": true,
  "data": {
    "newExecutionId": 1001,
    "status": "STARTED"
  },
  "error": null
}
```

10. API별 핵심 주의사항 요약

API	핵심 주의사항
주문 생성	재고 예약, 상태 PENDING_PAYMENT, 트랜잭션 범위

API	핵심 주의사항
결제 승인	Idempotency-Key, 금액 검증, 중복 승인 방지
PG 콜백	중복 콜백 허용, 이미 처리된 경우 성공 응답
주문 취소	배송 상태 확인, PG 취소, 재고 복구
리뷰 작성	실제 구매자 검증, UNIQUE 제약조건
상품 목록	DTO 조회, Keyset Pagination, 인덱스
관리자 검색	조회 조건별 인덱스, 개인정보 마스킹
배치 재실행	체크 처리, 실패 이력, 멍등성

11. 최종 정리

이 API 명세의 핵심은 다음과 같다.

1. 사용자 API와 관리자 API를 분리한다.
2. 주문 생성은 결제 완료가 아니라 결제 대기 상태를 만든다.
3. 결제 승인과 PG 콜백은 반드시 멍등하게 처리한다.
4. 주문 취소는 결제, 배송, 재고와 함께 처리한다.
5. 리뷰 작성은 실제 구매 여부와 중복 작성 여부를 검증한다.
6. 목록 조회는 Offset보다 Cursor 기반 페이징을 우선한다.
7. 운영자는 배치 실행 이력과 실패 데이터를 조회하고 재실행할 수 있어야 한다.

운영 가능한 API는 단순히 요청과 응답을 정의하는 것이 아니라, **상태 변경 규칙, 멍등성, 동시성, 장애 복구 기준**까지 포함해야 한다.